

SESSION 21

WEEK 21 — THREADS, RACE CONDITIONS AND CRITICAL SECTIONS

EXPERIMENT 21

Aim

To study **threads and multithreading**, understand race conditions and critical sections, and demonstrate synchronization using **POSIX threads (pthread)**.

Theory

A **thread** is the smallest unit of execution within a process. Multiple threads belonging to the same process share resources such as memory and files.

Process vs Thread

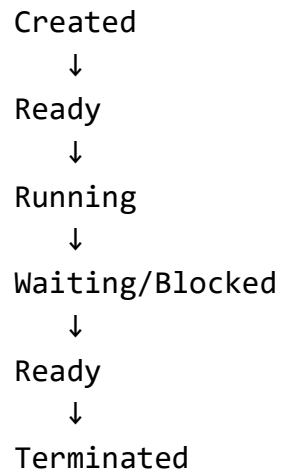
Process	Thread
Has its own address space	Shares address space with other threads
More expensive to create	Less expensive to create
Communication is comparatively complex	Communication is easier through shared memory
Process failure is generally isolated	A thread failure can affect the entire process

Thread Control Block (TCB)

The **TCB** stores information about a thread, such as:

- Thread ID
- Thread state
- Program counter
- CPU registers
- Stack information

Thread Lifecycle



Race Condition

A **race condition** occurs when multiple threads access shared data concurrently and the final result depends on the order in which the threads execute.

For example:

Thread 1 → reads counter
Thread 2 → reads counter
Thread 1 → updates counter
Thread 2 → updates counter

The expected updates can be lost.

Critical Section

A **critical section** is the part of a program where shared resources are accessed.

Only one thread should execute a critical section at a time when the shared resource requires mutual exclusion.

Mutex

A **mutex (mutual exclusion lock)** ensures that only one thread can enter a protected critical section at a time.

Algorithm

1. Create multiple threads using `pthread_create()`.
2. Give each thread a common shared variable.
3. Each thread increments the shared variable.
4. Protect the critical section using a mutex.
5. Wait for all threads using `pthread_join()`.
6. Display the final value.
7. Destroy the mutex.

C Program

```
C thread_mutex.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4
5  #define NUM_THREADS 4
6  #define ITERATIONS 100000
7
8  int counter = 0;
9
10 pthread_mutex_t lock;
11
12 void *increment_counter(void *arg)
13 {
14     for (int i = 0; i < ITERATIONS; i++) {
15
16         pthread_mutex_lock(&lock);
17
18         counter++;
19
20         pthread_mutex_unlock(&lock);
21     }
22
23     return NULL;
24 }
25
26 int main()
27 {
28     pthread_t threads[NUM_THREADS];
29
30     pthread_mutex_init(&lock, NULL);
31
32     printf("Creating %d threads...\n", NUM_THREADS);
33
34     for (int i = 0; i < NUM_THREADS; i++) {
35         pthread_create(&threads[i],
36                       NULL,
37                       increment_counter,
38                       NULL);
39     }
40
41     for (int i = 0; i < NUM_THREADS; i++) {
42         pthread_join(threads[i], NULL);
```

```

33
34     for (int i = 0; i < NUM_THREADS; i++) {
35         pthread_create(&threads[i],
36                        NULL,
37                        increment_counter,
38                        NULL);
39     }
40
41     for (int i = 0; i < NUM_THREADS; i++) {
42         pthread_join(threads[i], NULL);
43     }
44
45     printf("All threads completed.\n");
46     printf("Final counter value: %d\n", counter);
47
48     pthread_mutex_destroy(&lock);
49
50     return 0;
51 }

```

OUTPUT:

```

chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ gcc thread_mutex.c -o thread_mutex -pthread
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./thread_mutex
Creating 4 threads...
All threads completed.
Final counter value: 400000
chakrika@Chakrika:/mnt/c/Users/panak/Downloads/OSSP PRACTICALS$

```

Result

Thus, multithreading was successfully implemented using POSIX threads. The shared counter was protected using a mutex, preventing race conditions and ensuring the correct final counter value.

SESSION 22 — MUTEX AND CONDITION VARIABLES

EXPERIMENT 22

Aim

To study **mutex locks and condition variables** and implement the **Producer-Consumer problem** using POSIX threads for synchronization.

Theory

Mutex

A mutex provides **mutual exclusion**. It allows only one thread to access a shared resource at a time.

Important operations:

```
pthread_mutex_lock()  
pthread_mutex_unlock()
```

Condition Variable

A **condition variable** allows a thread to wait until a particular condition becomes true.

Important operations:

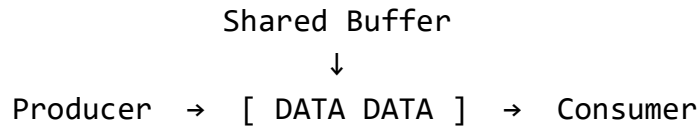
```
pthread_cond_wait()  
pthread_cond_signal()
```

Producer-Consumer Problem

The producer creates data and places it into a shared buffer.

The consumer removes data from the buffer.

The producer must wait when the buffer is full, while the consumer must wait when the buffer is empty.



Why Synchronization Is Needed

Without synchronization:

- Both threads could modify the buffer simultaneously.
- Data could be lost.
- The consumer could read an empty buffer.
- The producer could overwrite existing data.

Mutexes provide **mutual exclusion**, while condition variables provide **coordination between threads**.

Algorithm

Producer

1. Lock the mutex.
2. Check whether the buffer is full.
3. If full, wait using the condition variable.
4. Add an item to the buffer.
5. Signal the consumer.
6. Unlock the mutex.

Consumer

1. Lock the mutex.
2. Check whether the buffer is empty.
3. If empty, wait using the condition variable.

4. Remove an item from the buffer.
5. Signal the producer.
6. Unlock the mutex.

C Program

```
C producer_consumerr.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <unistd.h>
5
6  #define BUFFER_SIZE 5
7  #define ITEMS 10
8
9  int buffer[BUFFER_SIZE];
10
11 int in = 0;
12 int out = 0;
13 int count = 0;
14
15 pthread_mutex_t mutex;
16 pthread_cond_t not_full;
17 pthread_cond_t not_empty;
18
19 void *producer(void *arg)
20 {
21     for (int item = 1; item <= ITEMS; item++) {
22         pthread_mutex_lock(&mutex);
23
24         while (count == BUFFER_SIZE) {
25             pthread_cond_wait(&not_full, &mutex);
26         }
27
28         buffer[in] = item;
29         in = (in + 1) % BUFFER_SIZE;
30         count++;
31
32         printf("Produced: %d\n", item);
33
34         pthread_cond_signal(&not_empty);
35
36         pthread_mutex_unlock(&mutex);
37
38         usleep(100000);
39     }
40 }
```

```
41
42     return NULL;
43 }
44
45 void *consumer(void *arg)
46 {
47     for (int i = 1; i <= ITEMS; i++) {
48
49         pthread_mutex_lock(&mutex);
50
51         while (count == 0) {
52             pthread_cond_wait(&not_empty, &mutex);
53         }
54
55         int item = buffer[out];
56
57         out = (out + 1) % BUFFER_SIZE;
58         count--;
59
60         printf("Consumed: %d\n", item);
61
62         pthread_cond_signal(&not_full);
63
64         pthread_mutex_unlock(&mutex);
65
66         usleep(150000);
67     }
68
69     return NULL;
70 }
71
72 int main()
73 {
74     pthread_t producer_thread;
75     pthread_t consumer_thread;
```

```

69     return NULL;
70 }
71
72 int main()
73 {
74     pthread_t producer_thread;
75     pthread_t consumer_thread;
76
77     pthread_mutex_init(&mutex, NULL);
78     pthread_cond_init(&not_full, NULL);
79     pthread_cond_init(&not_empty, NULL);
80
81     printf("Starting Producer-Consumer simulation...\n\n");
82
83     pthread_create(&producer_thread,
84                   NULL,
85                   producer,
86                   NULL);
87
88     pthread_create(&consumer_thread,
89                   NULL,
90                   consumer,
91                   NULL);
92
93     pthread_join(producer_thread, NULL);
94     pthread_join(consumer_thread, NULL);
95
96     pthread_mutex_destroy(&mutex);
97     pthread_cond_destroy(&not_full);
98     pthread_cond_destroy(&not_empty);
99
100    printf("\nProducer and Consumer completed successfully.\n");
101
102    return 0;
103 }

```

OUTPUT:

```
chakrika@Chakrika: /mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ gcc producer_consumerr.c -o producer_consumerr -pthread
chakrika@Chakrika: /mnt/c/Users/panak/Downloads/OSSP PRACTICALS$ ./producer_consumerr
Starting Producer-Consumer simulation...

Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Produced: 5
Consumed: 4
Produced: 6
Consumed: 5
Produced: 7
Produced: 8
Consumed: 6
Produced: 9
Consumed: 7
Produced: 10
Consumed: 8
Consumed: 9
Consumed: 10

Producer and Consumer completed successfully.
chakrika@Chakrika: /mnt/c/Users/panak/Downloads/OSSP PRACTICALS$
```

Result

Thus, the Producer-Consumer problem was successfully implemented using POSIX threads, mutex locks, and condition variables. The mutex provided mutual exclusion, while condition variables synchronized the producer and consumer threads and prevented access to an empty or full buffer.